

Discovering the Output Pattern of AND Gates in Boolean Networks

June 18, 2025

Abstract

This document unveils a mathematical formula for pinpointing the indices where an AND gate's output is 1 in a Boolean network. We begin with an intuitive exploration, followed by a rigorous definition, data structure explanation, and a Mathematica formulation. The derived expression, validated through intuitive examples, offers a non-iterative solution, mirroring the elegance of classical physics equations.

1 Developing Intuition

Picture a network of light bulbs, each controlled by multiple switches wired through an AND gate—only when all switches are flipped ON (1) does the bulb light up (output 1). The switches are interconnected, and we need to determine when this bulb shines across all possible configurations. With many bulbs and switches, testing every combination manually is impractical, especially for large networks.

Consider a small setup with three switches controlling one bulb via an AND gate. The possible states—(0,0,0), (0,0,1), ..., (1,1,1)—are listed in order of their binary value. The bulb lights only when all three switches are ON, like (1,1,1). But in a larger network, some switches might not directly affect this bulb, acting as background noise. We notice that the bulb's ON states cluster in predictable spots, starting after a certain point determined by the key switches, with additional shifts based on the background switches.

This is like tuning a radio to a station—key switches set the frequency (a baseline), and background switches fine-tune the signal. By understanding this rhythm, we can calculate the exact moments the bulb lights without checking every setting. The rest of this document turns this intuition into a precise formula, adaptable to any network size.

2 Problem Definition

Consider a Boolean network with n nodes, represented by an $n \times n$ adjacency matrix C , where $C_{k,i} = 1$ if node k receives input from node i , and $C_{k,i} = 0$ otherwise. The dynamics list D specifies each node's Boolean function, with node k using AND. The input repertoire consists of 2^n binary vectors $v = (v_1, v_2, \dots, v_n)$, where $v_i \in \{0, 1\}$, indexed from 1 to 2^n , with each index j corresponding to the binary representation of $j - 1$ (0-based decimal).

The task is to find all indices $j \in \{1, 2, \dots, 2^n\}$ where the output of node k is 1, given that the AND function requires $v_i = 1$ for all $i \in I_c$, where $I_c = \{i \mid C_{k,i} = 1\}$ is the set of connected input nodes.

3 Structure. Explanation of the Structure of the Data

The input repertoire is a sequence of 2^n binary vectors, ordered by their decimal value (1-based). For $n = 3$, the repertoire is:

Index	Binary Vector	Decimal (0-based)	Node 1 Output (AND)
1	(0, 0, 0)	000	0
2	(1, 0, 0)	001	0
3	(0, 1, 0)	010	0
4	(1, 1, 0)	011	0
5	(0, 0, 1)	100	0
6	(1, 0, 1)	101	0
7	(0, 1, 1)	110	1
8	(1, 1, 1)	111	1

If node 1 uses AND with inputs from nodes 2 and 3 ($I_c = \{2, 3\}$), the output is 1 at indices 7 and 8, which are exactly the two indices where $v_2 = v_3 = 1$.

Correction, 2026-09-03 (AUDIT03/R0). The index-7 entry of this table previously read 0. That value tabulated AND over $I_c = \{1, 2, 3\}$, not over the $I_c = \{2, 3\}$ stated in the surrounding text, and it was the sole such entry. The error is decidable without reference to any bit-ordering convention: an AND over d connected inputs has one-set size exactly 2^{n-d} , because the $n - d$ coordinates it never reads are free; at $n = 3$, $d = 2$ that is 2, whereas the old column contained a single 1. A count is invariant under relabelling, so no ordering could reconcile it — checked exhaustively over all six coordinate labellings and all three two-subsets, of which none reproduced the old column. The ordering used throughout is LSB-first, canonical per GOVERNANCE/ORDERING.md §1 and stated in both manuscripts. The structure hinges on connected nodes' contributions, forming a baseline index (the decimal anchor P , GOVERNANCE/GLOSSARY.md sec.1a), while disconnected nodes ($I_{nc} = \{1\}$) shift this baseline.

The decimal value of a vector v is:

$$in_{dec} = \sum_{i=1}^n v_i \cdot 2^{i-1}$$

Connected nodes' decimal is $in_{c_{dec}} = \sum_{i \in I_c} v_i \cdot 2^{i-1}$, and disconnected nodes' offset is $\Delta_{nc} = \sum_{i \in I_{nc}} v_i \cdot 2^{i-1}$, with $in_{dec} = in_{c_{dec}} + \Delta_{nc}$.

4 Mathematica Formulation

The indices J_k where node k 's AND output is 1 are computed as:

$$J_k = P + 1 + \Delta_{nc}$$

where: - $P = \sum_{i \in I_c} 2^{i-1}$ is the decimal anchor, the sum of decimal contributions from connected nodes. - $\Delta_{nc} = \sum_{i \in I_{nc}} v_i \cdot 2^{i-1}$ is the offset from disconnected nodes' states, with $v_i \in \{0, 1\}$. - $I_c = \{i \mid C_{k,i} = 1\}$ and $I_{nc} = \{1, 2, \dots, n\} \setminus I_c$. - $J_k \leq 2^n$ to stay within the repertoire.

5 Ordering and Network-Aware Constraint

Ordering policy: Inputs are enumerated by the binary representation of $j - 1$ over length n . Two orderings are relevant:

- **LSB-first** (code default): $v(j) = \text{ReverseIntegerDigits}(j - 1, 2, n)$, weights 2^{i-1} .
- **MSB-first** (manuscript): $v(j) = \text{IntegerDigits}(j - 1, 2, n)$, weights 2^{n-i} .

The bit-reversal map $\varphi(j) = 1 + \text{FromDigits}(\text{Reverse}(\text{IntegerDigits}(j - 1, 2, n)), 2)$ is an involution and relates index sets between orderings.

Network-aware constraint: The candidate form $P + 1 + \Delta_{nc}$ must be restricted to indices j whose connected bits satisfy $v_i(j) = 1$ for all $i \in I_c$. Hence the valid one-set is

$$J_k = \left\{ j \in \{1, \dots, 2^n\} \mid \forall i \in I_c, v_i(j) = 1 \right\}, \quad v(j) \text{ ordered per policy.}$$

Under LSB-first and weights 2^{i-1} , this equals the filtered subset of $\{P + 1 + \Delta_{nc}\}$; under MSB-first it equals φ -mapped indices of the same construction.

6 Explanation of the Mathematical Formulation

Let’s break down the formula step-by-step, imagining we’re tuning a system to find when the AND gate activates.

1. **Set the Anchor with Connected Nodes:** The decimal anchor P is the value when all connected inputs are 1. For $I_c = \{2, 3\}$, $P = 2^{2-1} + 2^{3-1} = 2 + 4 = 6$. This is the starting point after which 1s can occur.
2. **Adjust with Disconnected Nodes:** The offset Δ_{nc} comes from the binary states of disconnected nodes. If $I_{nc} = \{1\}$, $\Delta_{nc} = v_1 \cdot 2^{1-1} = v_1 \cdot 1$, so Δ_{nc} is 0 or 1.
3. **Compute the Index:** Adding 1 (for 1-based indexing) and Δ_{nc} to P gives J_k . For $P = 6$, $\Delta_{nc} = 0$ (if $v_1 = 0$) yields $J_k = 6 + 1 + 0 = 7$, and $\Delta_{nc} = 1$ (if $v_1 = 1$) yields 8. Only $j = 8$ (1,1,1) satisfies the AND condition.
4. **Ensure Validity:** The formula assumes $v_i = 1$ for $i \in I_c$, and Δ_{nc} explores all $2^{n-|I_c|}$ combinations of disconnected nodes, naturally filtering valid 1s.

This method uses the network’s structure to predict 1s, avoiding brute-force checks by leveraging the additive nature of binary decimals.

7 Examples (intuitive)

For $n = 3$, node 1 with $I_c = \{2, 3\}$ (AND): - $P = 2 + 4 = 6$, $I_{nc} = \{1\}$, $\Delta_{nc} = v_1 \cdot 1$. - $J_1 = 6 + 1 + 0 = 7$ (0,1,1), $J_1 = 6 + 1 + 1 = 8$ (1,1,1). - Check: both 7 and 8 satisfy $v_2 = v_3 = 1$, so $J_1 = \{7, 8\}$ is exactly the AND one-set. Verified by exhaustive evaluation, 2026-09-03.

For $n = 7$, node 4 with $I_c = \{1, 3, 5, 7\}$: - $P = 1 + 4 + 16 + 64 = 85$, $I_{nc} = \{2, 4, 6\}$, $\Delta_{nc} = v_2 \cdot 2 + v_4 \cdot 8 + v_6 \cdot 32$. - $\Delta_{nc} \in \{0, 2, 8, 10, 32, 34, 40, 42\}$, the subset sums of the weights $\{2, 8, 32\}$, so $J_4 = \{86, 88, 94, 96, 118, 120, 126, 128\}$, of size $2^{7-4} = 8$. - Verified identical to exhaustive evaluation, 2026-09-03. (The earlier remark that the ones “align when $\Delta_{nc} = 1, 3, 5, \dots$ ” was wrong: every weight in I_{nc} is even here, so no odd offset arises.)

8 Discussion

The formula $J_k = P + 1 + \Delta_{nc}$ echoes $F = ma$, with P as the fixed mass (connected nodes’ influence) and Δ_{nc} as the variable acceleration (disconnected nodes’ tuning). It captures causality—1s depend solely on connected nodes being 1, with disconnected nodes shifting the pattern.

Withdrawn, 2026-09-03 (AUDIT03/R0). This section previously concluded that Δ_{nc} “may overgenerate (e.g., index 7 in the 3-node case)” and needed further constraint. It does not. That concern was an artefact of the single incorrect table entry corrected in Section 3, and of nothing else. With the entry corrected, $J_k = P + 1 + \Delta_{nc}$ reproduces the AND one-set exactly, both for the 3-node example and for the 7-node example above, each checked against exhaustive evaluation. No refinement of Δ_{nc} is required.

9 Conclusion

We’ve derived $J_k = P + 1 + \Delta_{nc}$ as a non-iterative formula for AND gate 1s, grounded in intuition and structure. It is exact as stated: on both worked examples the formula reproduces the exhaustive AND one-set elementwise, and the family of offsets Δ_{nc} is precisely the set of subset sums of the weights of the coordinates the gate does not read.